

LAPORAN PROGRES
MINGGU KE – 3
Mata Kuliah Secure Software Engineering



Dosen Pengampu :
Septian Geges, S.Kom., M.Kom.

Disusun Oleh :

Hessel Josef Imanuel	2330205030062
Apreldiovano Brian Walmaputra	2330305030089
Rafael Sanjaya	2330305030075
Yohanes Abel Kelvin	2330205030051

JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PALANGKA RAYA
2026

DAFTAR ISI

DAFTAR ISI	i
BAB I PENDAHULUAN	1
1.1 Gambaran Umum Proyek	1
1.2 Tujuan	1
1.3 Ruang Lingkup.....	2
BAB II PEMBAHASAN	3
2.1 Lingkup Pengujian Keamanan.....	3
2.2 Pengujian SAST Menggunakan Semgrep.....	4
2.3 Pengujian DAST Menggunakan OWASP ZAP	6
2.4 Manual Security Testing	9
2.5 Pengujian SQL Injection.....	10
2.6 Pengujian Rate Limiting Login.....	11
2.7 Pengujian CSRF Protection	12
2.8 Pengujian Session dan Cookie	12
2.9 Pengujian RBAC dan IDOR	13
2.10 Pengujian XSS	13
2.11 Perbaikan Temuan Keamanan.....	14
2.12 CVSS dan Risk Rating.....	16
2.13 Vulnerability Report.....	17
2.14 Source Code Hasil Perbaikan.....	18
2.15 Executive Summary Postur Keamanan.....	19
2.16 Hambatan dan Solusi	19
2.17 Rencana Minggu 4	20
BAB III KESIMPULAN.....	21
3.1 Kesimpulan	21
3.2 Saran	21

BAB I

PENDAHULUAN

1.1 Gambaran Umum Proyek

Pada minggu ketiga proyek Secure Software Engineering, kegiatan pengembangan sistem LuxeBooking difokuskan pada tahap pengujian keamanan atau security testing. Tahap ini dilakukan setelah pada minggu sebelumnya sistem telah mengimplementasikan beberapa fitur keamanan inti, seperti autentikasi, manajemen sesi, Role-Based Access Control, CSRF protection, validasi input, password hashing, rate limiting, security headers, dan pencegahan Cross-Site Scripting.

Pengujian keamanan diperlukan untuk memastikan bahwa kontrol keamanan yang telah diterapkan tidak hanya tersedia pada source code, tetapi juga benar-benar berjalan ketika aplikasi digunakan. Pada tahap ini, pengujian dilakukan melalui beberapa pendekatan, yaitu Static Application Security Testing, Dynamic Application Security Testing, manual security testing, penilaian risiko menggunakan CVSS, serta penyusunan vulnerability report.

Static Application Security Testing dilakukan untuk menganalisis source code tanpa menjalankan aplikasi. Dynamic Application Security Testing dilakukan untuk menguji aplikasi saat sedang berjalan menggunakan OWASP ZAP. Selain itu, pengujian manual dilakukan untuk memeriksa keamanan autentikasi, otorisasi, manajemen sesi, CSRF, SQL Injection, IDOR, XSS, cookie, error handling, dan security headers.

Berdasarkan hasil pengujian, beberapa temuan keamanan berhasil ditemukan, seperti .env Information Leak, Directory Browsing, dan Missing Security Headers. Temuan tersebut kemudian dianalisis, diberi tingkat risiko, dan diperbaiki melalui konfigurasi .htaccess. Dengan demikian, progress minggu ketiga tidak hanya berfokus pada menemukan celah keamanan, tetapi juga melakukan perbaikan terhadap temuan prioritas.

1.2 Tujuan

Tujuan dari laporan progress minggu ketiga adalah sebagai berikut:

1. Melakukan pengujian SAST menggunakan Semgrep.
2. Melakukan pengujian DAST menggunakan OWASP ZAP.
3. Melakukan manual security testing pada fitur-fitur penting sistem.
4. Mengidentifikasi temuan keamanan dari hasil SAST, DAST, dan manual testing.

5. Memberikan CVSS score pada setiap temuan keamanan.
6. Menyusun vulnerability report dengan format profesional.
7. Melakukan perbaikan terhadap temuan prioritas.
8. Mendokumentasikan source code hasil perbaikan melalui commit GitHub.
9. Menyusun rencana tindak lanjut untuk minggu keempat.

1.3 Ruang Lingkup

Ruang lingkup kegiatan minggu ketiga meliputi pengujian keamanan pada sistem LuxeBooking yang berjalan pada lingkungan lokal menggunakan XAMPP. Komponen yang diuji meliputi frontend, backend, database, konfigurasi server lokal, dan mekanisme keamanan aplikasi.

Pengujian dilakukan pada file frontend seperti login.html, admin.html, dan customer.html, serta file backend seperti api.php dan config.php. Selain itu, pengujian juga dilakukan terhadap konfigurasi .env, .htaccess, security headers, cookie, session, dan akses role pengguna.

BAB II

PEMBAHASAN

2.1 Lingkup Pengujian Keamanan

Pengujian keamanan dilakukan untuk mengevaluasi sistem LuxeBooking dari sisi source code, aplikasi yang berjalan, dan perilaku sistem terhadap input atau request yang berpotensi berbahaya. Lingkup pengujian disusun agar setiap bagian penting sistem dapat diperiksa secara terstruktur.

Pengujian mencakup tiga pendekatan utama, yaitu SAST untuk memeriksa source code, DAST untuk menguji aplikasi yang berjalan, dan manual testing untuk memvalidasi skenario keamanan yang mungkin tidak sepenuhnya terdeteksi oleh tools otomatis. Selain itu, setiap temuan didokumentasikan dalam vulnerability report dan diberi CVSS score.

Tabel 2.1 Lingkup Pengujian Keamanan

No	Komponen	Bagian yang Diuji	Keterangan
1	Frontend	login.html, admin.html, customer.html	Pengujian tampilan, role access, XSS, dan cookie
2	Backend	api.php, config.php	Pengujian autentikasi, CSRF, session, validasi, dan API
3	Database	users, sessions, bookings, login_attempts	Pengujian session, booking, dan rate limiting
4	Konfigurasi	.env, .htaccess, security headers	Pengujian file sensitif dan konfigurasi server
5	Tools	Semgrep, OWASP ZAP, Chrome DevTools	Digunakan untuk SAST, DAST, dan manual testing

Berdasarkan Tabel 2.1, pengujian tidak hanya dilakukan pada fitur aplikasi, tetapi juga pada konfigurasi server lokal. Hal ini penting karena beberapa temuan keamanan seperti .env Information Leak dan Directory Browsing tidak berasal dari kesalahan logika program, tetapi dari konfigurasi server yang belum dibatasi.

2.2 Pengujian SAST Menggunakan Semgrep

Static Application Security Testing dilakukan menggunakan Semgrep. Pengujian ini bertujuan untuk menemukan potensi kerentanan pada source code tanpa perlu menjalankan aplikasi. Dengan SAST, tim dapat mengetahui apakah terdapat pola kode yang berisiko, seperti hardcoded secret, potensi SQL Injection, output tanpa escaping, atau konfigurasi yang kurang aman.

Perintah yang digunakan pada pengujian SAST adalah sebagai berikut:

semgrep --config=p/owasp-top-ten .

```
PS C:\xampp\htdocs\hotel-booking-secure-system> semgrep --config=p/owasp-top-ten .

Semgrep CLI

Loading rules from registry...
Scanning 16 files (only git-tracked) with 544 Code rules:

CODE RULES

Language      Rules  Files  Origin  Rules
-----
<multilang>    6     16    Community  544
php            22     3
html           1     3

SUPPLY CHAIN RULES

Sign in with `semgrep login` and run
`semgrep ci` to find dependency vulnerabilities and
advanced cross-file findings.

PROGRESS

100% 0:00:00
```

```
3 Code Findings

backend\api.php
>>> php.lang.security.injection.echoed-request.echoed-request
    Blocking
    Echo'ing user input risks cross-site scripting vulnerability. You should use
    `htmlentities()` when showing data to users.
    Details: https://sg.run/Bqqb

    Autofix ▶ echo htmlentities(json_encode(['success' => true, 'e_voucher' => $voucher]));
424: echo json_encode(['success' => true, 'e_voucher' => $voucher]);

backend\config.php
>>> generic.nginx.security.request-host-used.request-host-used
    Blocking
    '$http_host' and '$host' variables may contain a malicious value from attacker controlled
    'Host' request header. Use an explicitly configured host value or a allow list for
    validation.
    Details: https://sg.run/4x3Z

28: $host      = 'localhost';
...
34: $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username,
    $password, [

Scan Summary
```

```

✔ Scan completed successfully.
• Findings: 3 (3 blocking)
• Rules run: 29
• Targets scanned: 16
• Parsed lines: ~99.9%
• Scan skipped:
  ◦ Files larger than files 1.0 MB: 3
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 29 rules on 16 files: 3 findings.
💡 Missed out on 1803 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
PS C:\xampp\htdocs\hotel-booking-secure-system>

```

Gambar 2.1 Hasil Pengujian SAST Menggunakan Semgrep

Berdasarkan Gambar 2.1, Semgrep berhasil melakukan pemindaian terhadap source code proyek LuxeBooking. Hasil pengujian menunjukkan terdapat beberapa temuan yang perlu dianalisis. Temuan tersebut tidak langsung dianggap sebagai kerentanan valid, tetapi perlu direview untuk menentukan apakah termasuk temuan nyata, false positive, atau accepted risk.

Tabel 2.2 Hasil Pengujian SAST

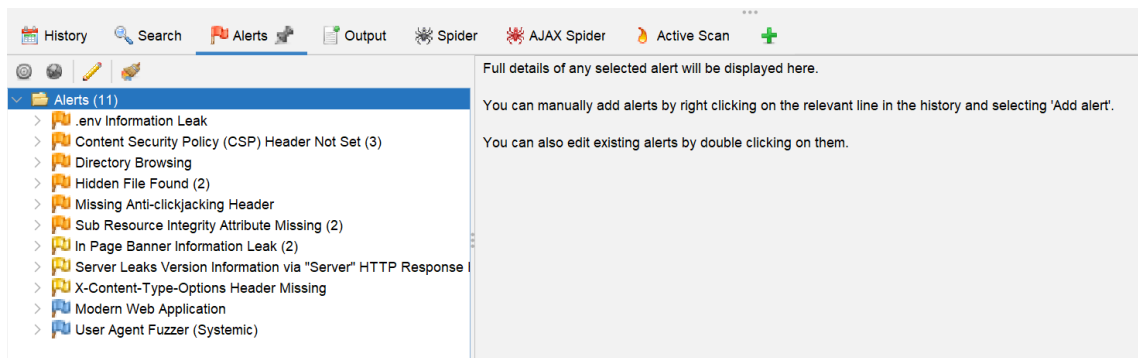
ID	File	Temuan	Severity	Status	Keterangan
SAST-001	backend/api.php	Potensi output tanpa escaping pada response JSON	Blocking	Accepted Risk	Response berupa JSON API dan tidak langsung merender HTML
SAST-002	backend/config.php	Variabel \$host terdeteksi sebagai potensi host issue	Blocking	False Positive	\$host digunakan untuk koneksi database lokal, bukan HTTP Host header
SAST-003	backend/api.php	Potensi echoed request	Blocking	Reviewed	Frontend sudah menggunakan escapeHTML() untuk output data

Berdasarkan Tabel 2.2, tidak semua temuan SAST merupakan kerentanan yang valid. Temuan pada config.php dikategorikan sebagai false positive karena variabel \$host digunakan untuk konfigurasi database MySQL dengan nilai statis localhost. Temuan pada api.php dikategorikan sebagai accepted risk karena response dikembalikan dalam bentuk JSON API, bukan HTML langsung. Meskipun demikian, temuan tersebut tetap didokumentasikan sebagai bagian dari proses secure code review.

2.3 Pengujian DAST Menggunakan OWASP ZAP

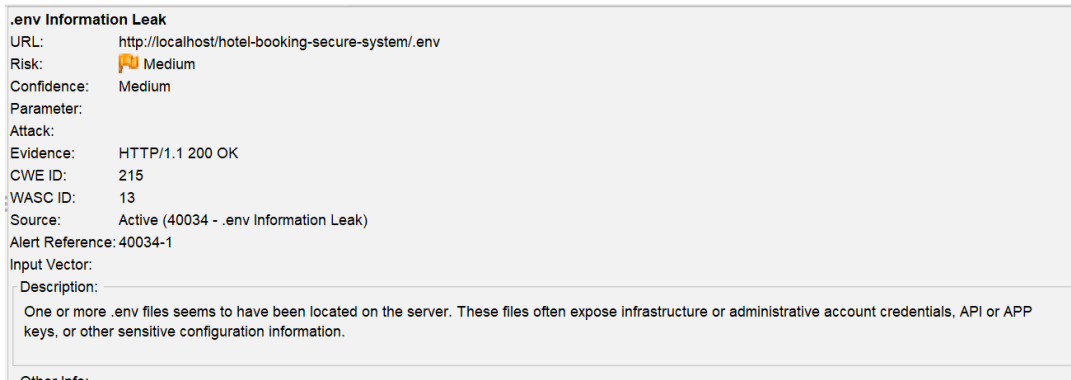
Dynamic Application Security Testing dilakukan menggunakan OWASP ZAP terhadap aplikasi LuxeBooking yang berjalan pada localhost. Berbeda dengan SAST yang memeriksa source code, DAST menguji aplikasi yang sedang berjalan dengan mengirim request dan payload tertentu untuk menemukan potensi kerentanan.

Target pengujian OWASP ZAP adalah url `http://localhost/hotel-booking-secure-system/frontend/login.html`. Pengujian DAST dilakukan melalui fitur Automated Scan pada OWASP ZAP. Proses pengujian mencakup spider atau crawling untuk menemukan halaman dan endpoint, kemudian active scan untuk mengirim payload pengujian ke aplikasi.



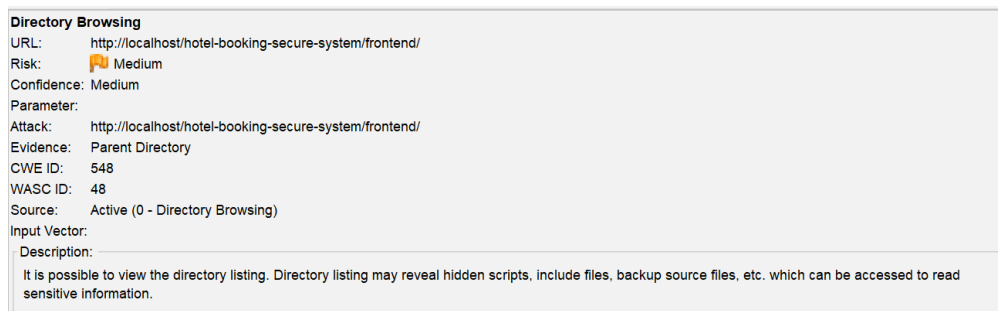
Gambar 2.2 Hasil Alert Pengujian DAST Menggunakan OWASP ZAP

Berdasarkan Gambar 2.2, OWASP ZAP menemukan beberapa alert pada aplikasi LuxeBooking. Temuan utama yang perlu diperhatikan adalah .env Information Leak, Directory Browsing, dan Missing Security Headers. Ketiga temuan tersebut menjadi prioritas perbaikan karena dapat berdampak pada kebocoran informasi dan lemahnya perlindungan browser.



Gambar 2.3 Detail Alert .env Information Leak

Gambar 2.3 menunjukkan bahwa file .env dapat diakses melalui browser dengan response berhasil. Kondisi ini berisiko karena file .env berisi konfigurasi dan credential sensitif. Oleh karena itu, temuan ini dikategorikan sebagai valid dan menjadi prioritas perbaikan.



Gambar 2.4 Detail Alert Directory Browsing

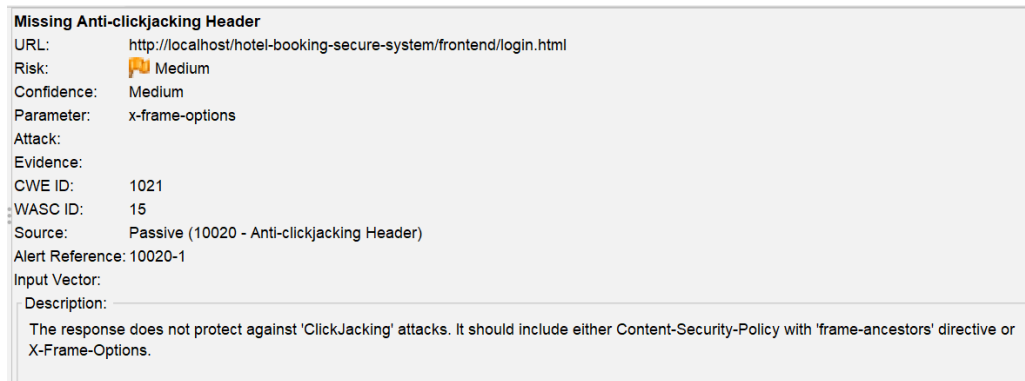
Gambar 2.4 menunjukkan bahwa folder /frontend/ dapat menampilkan daftar file seperti login.html, admin.html, dan customer.html. Directory Browsing dapat membantu pihak tidak berwenang memahami struktur aplikasi, sehingga perlu dinonaktifkan melalui konfigurasi server.



Gambar 2.5 Detail Alert Content Security Policy Header Not Set

Gambar 2.5 menunjukkan bahwa OWASP ZAP mendeteksi belum adanya header Content Security Policy pada file statis. Header CSP diperlukan untuk membatasi sumber

konten yang boleh dimuat oleh browser, seperti script, style, font, gambar, dan koneksi eksternal.



Gambar 2.6 Detail Alert Missing Anti-clickjacking Header

Gambar 2.6 menunjukkan bahwa header anti-clickjacking belum diterapkan pada halaman yang diuji. Header ini diperlukan untuk mengurangi risiko halaman dimuat di dalam frame oleh situs lain.

Tabel 2.3 Hasil Pengujian DAST OWASP ZAP

ID	Alert	Risk	Status Awal	Keterangan
DAST-001	.env Information Leak	Medium	Valid	File .env dapat diakses melalui browser
DAST-002	Directory Browsing	Medium	Valid	Folder /frontend/ menampilkan daftar file
DAST-003	Content Security Policy Header Not Set	Medium	Valid	CSP belum muncul pada file statis
DAST-004	Missing Anti-clickjacking Header	Medium	Valid	Header X-Frame-Options belum muncul
DAST-005	X-Content-Type-Options Header Missing	Low	Valid	Header nosniff belum muncul

DAST-006	Server Version Disclosure	Low	Planned Fix	Versi Apache/PHP terlihat pada response header
DAST-007	Sub Resource Integrity Missing	Low	Accepted Risk	CDN eksternal belum memakai SRI

Berdasarkan Tabel 2.3, temuan dengan prioritas utama adalah .env Information Leak, Directory Browsing, dan Missing Security Headers. Temuan lain seperti Server Version Disclosure dan Sub Resource Integrity Missing tetap dicatat, tetapi direncanakan untuk tahap hardening berikutnya.

2.4 Manual Security Testing

Manual security testing dilakukan untuk memvalidasi keamanan sistem melalui percobaan langsung pada aplikasi. Pengujian manual diperlukan karena tidak semua skenario keamanan dapat ditemukan oleh tools otomatis. Pada tahap ini, pengujian dilakukan terhadap autentikasi, otorisasi, IDOR, session, CSRF, SQL Injection, XSS, cookie, error handling, dan security headers.

Tabel 2.4 Hasil Manual Security Testing

No	Pengujian	Langkah Pengujian	Hasil	Status
1	SQL Injection	Input payload ' OR 1=1 -- pada form login	Login gagal	Passed
2	Rate Limiting	Memasukkan password salah beberapa kali	Sistem membatasi percobaan login	Passed
3	CSRF Protection	Menghapus csrf_token, lalu melakukan aksi penting	Request ditolak	Passed
4	Session Logout	Logout lalu membuka dashboard kembali	User diarahkan ke login	Passed

5	RBAC	Customer membuka halaman admin	Customer diarahkan ke login	Passed
6	IDOR	Customer B memeriksa booking Customer A	Data tidak muncul	Passed
7	XSS	Input payload <script>alert(1)</script>	Payload tidak dieksekusi	Passed
8	Cookie Security	Cek cookie pada DevTools	auth_token HttpOnly dan SameSite=Lax	Passed
9	Error Handling	Input data tidak valid	Error tidak membocorkan detail sistem	Passed
10	Security Headers	Cek response header pada login.html	Header muncul setelah perbaikan	Fixed

Berdasarkan Tabel 2.4, sebagian besar mekanisme keamanan yang telah diterapkan pada minggu kedua berjalan dengan baik saat diuji secara manual. Pengujian SQL Injection, CSRF, RBAC, IDOR, XSS, dan cookie menunjukkan hasil Passed. Temuan pada security headers awalnya belum sepenuhnya aman, tetapi telah diperbaiki melalui file .htaccess.

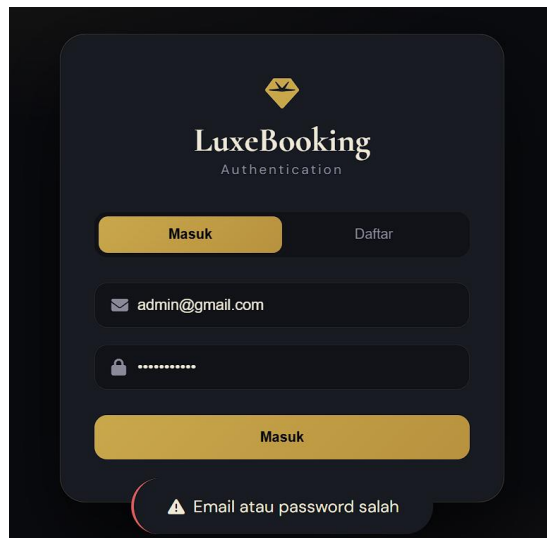
2.5 Pengujian SQL Injection

Pengujian SQL Injection dilakukan pada form login dengan memasukkan payload berikut:

Email : admin@gmail.com

Password : ' OR '1'='1

Payload tersebut digunakan untuk menguji apakah sistem dapat dilewati dengan manipulasi query SQL. Jika sistem rentan terhadap SQL Injection, payload tersebut berpotensi membuat login berhasil tanpa kredensial yang benar.

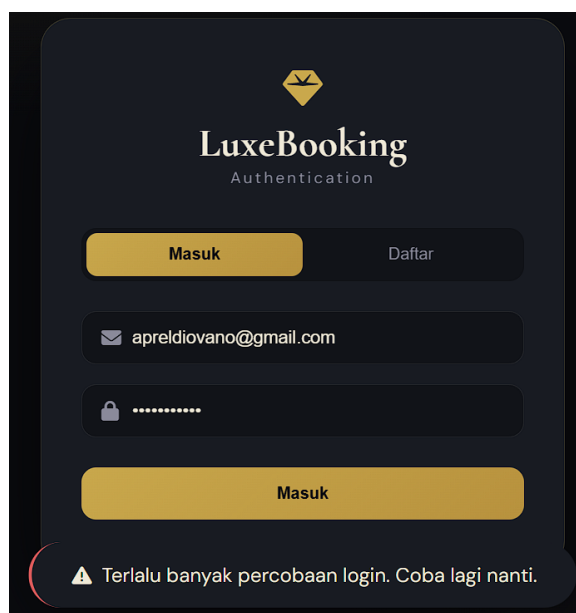


Gambar 2.7 Pengujian SQL Injection pada Form Login

Berdasarkan Gambar 2.7, payload SQL Injection tidak berhasil melewati proses login. Sistem tetap menolak autentikasi dan tidak memberikan akses ke dashboard. Hal ini menunjukkan bahwa penggunaan prepared statement pada backend telah membantu mencegah serangan SQL Injection.

2.6 Pengujian Rate Limiting Login

Pengujian rate limiting dilakukan dengan memasukkan password yang salah secara berulang pada form login. Pengujian ini bertujuan untuk mengetahui apakah sistem membatasi percobaan login gagal agar tidak mudah diserang menggunakan brute force.

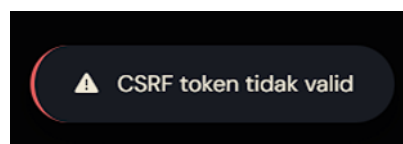


Gambar 2.8 Pengujian Rate Limiting pada Login

Berdasarkan Gambar 2.8, sistem menolak login setelah beberapa kali percobaan gagal. Hal ini menunjukkan bahwa mekanisme rate limiting telah berjalan. Dengan adanya pembatasan percobaan login, risiko brute force terhadap akun pengguna dapat dikurangi.

2.7 Pengujian CSRF Protection

Pengujian CSRF dilakukan dengan menghapus cookie csrf_token melalui Chrome DevTools, kemudian mencoba menjalankan aksi penting seperti booking, tambah kamar, atau hapus data. Tujuannya adalah untuk memastikan bahwa request penting tidak dapat diproses tanpa CSRF token yang valid.



Gambar 2.9 Pengujian CSRF Token Tidak Valid

Berdasarkan Gambar 2.9, sistem menolak request ketika CSRF token tidak tersedia atau tidak cocok. Hal ini menunjukkan bahwa mekanisme CSRF protection sudah berjalan. Request penting harus membawa token yang sesuai antara cookie csrf_token dan header X-CSRF-Token.

2.8 Pengujian Session dan Cookie

Pengujian session dilakukan dengan melakukan login, logout, kemudian mencoba membuka kembali halaman dashboard. Jika session berhasil dihapus, maka user tidak boleh dapat mengakses dashboard setelah logout.

Pengujian cookie dilakukan melalui Chrome DevTools pada bagian Application dan Cookies. Pengujian ini bertujuan untuk memastikan bahwa cookie autentikasi memiliki atribut keamanan yang sesuai.

auth_token	ebdd609609630fe79fb9c95f7d73f6276ea2...	localh...	/	2026-...	74	✓		Lax		Medi...
csrf_token	57f0a2c738dfb64857090a8a7537cc530996...	localh...	/	2026-...	74			Lax		Medi...
...	...	...	/	...	...	...	...	...	...	...

Gambar 2.10 Pengujian Cookie HttpOnly dan SameSite

Masukkan screenshot cookie auth_token dan csrf_token pada DevTools.

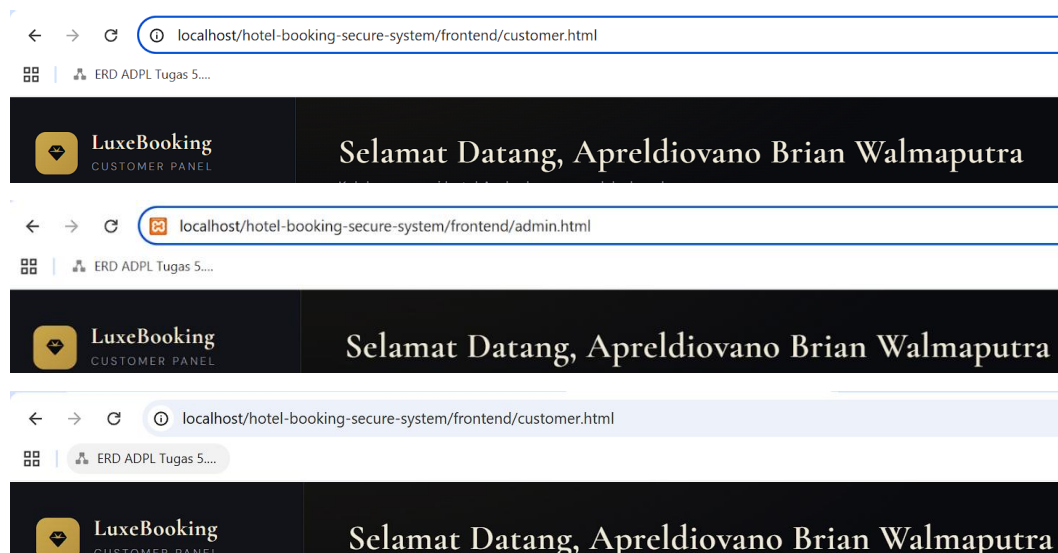
Berdasarkan Gambar 2.10, cookie auth_token telah menggunakan atribut HttpOnly dan SameSite=Lax. Atribut HttpOnly membuat token tidak dapat dibaca langsung oleh JavaScript, sedangkan SameSite=Lax membantu mengurangi risiko CSRF.

Cookie `csrf_token` tidak dibuat `HttpOnly` karena perlu dibaca oleh JavaScript untuk dikirim melalui header `X-CSRF-Token`.

2.9 Pengujian RBAC dan IDOR

Pengujian RBAC dilakukan dengan login sebagai customer, kemudian mencoba membuka halaman admin. Tujuan pengujian ini adalah memastikan bahwa pengguna dengan role customer tidak dapat mengakses fitur yang hanya boleh digunakan oleh admin.

Pengujian IDOR dilakukan dengan membuat booking menggunakan Customer A, kemudian login sebagai Customer B dan memeriksa menu booking. Jika sistem aman, Customer B tidak boleh melihat booking milik Customer A.



Gambar 2.11 Pengujian Role-Based Access Control Customer ke Admin

Berdasarkan Gambar 2.11, customer tidak dapat mengakses halaman admin dan diarahkan kembali ke halaman login. Hal ini menunjukkan bahwa Role-Based Access Control telah berjalan. Selain itu, hasil pengujian IDOR menunjukkan bahwa customer hanya dapat melihat data booking miliknya sendiri.

2.10 Pengujian XSS

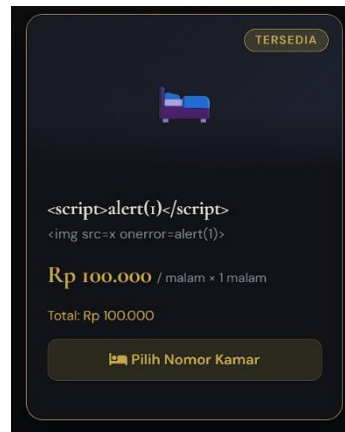
Pengujian Cross-Site Scripting dilakukan dengan memasukkan payload pada form edit kamar. Payload yang digunakan adalah:

```
<script>alert(1)</script>
```

dan:

```
<img src=x onerror=alert(1)>
```

Pengujian ini bertujuan untuk mengetahui apakah data yang dimasukkan pengguna dapat dieksekusi sebagai script oleh browser ketika ditampilkan kembali di halaman admin atau customer.



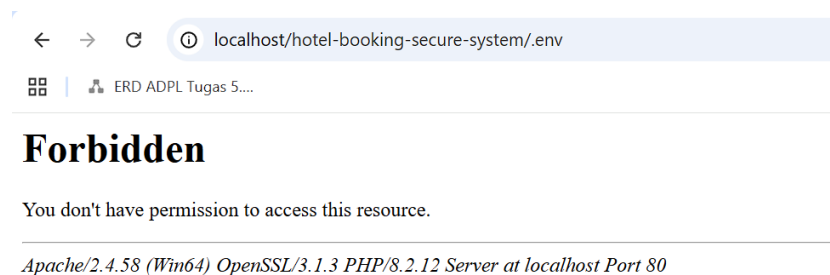
Gambar 2.12 Hasil Pengujian XSS pada Halaman Customer

Berdasarkan Gambar 2.12, payload XSS tidak dieksekusi oleh browser. Payload hanya tampil sebagai teks biasa pada halaman customer. Dengan demikian, pengujian XSS dinyatakan Passed karena sistem tidak menjalankan script berbahaya pada output data kamar.

2.11 Perbaikan Temuan Keamanan

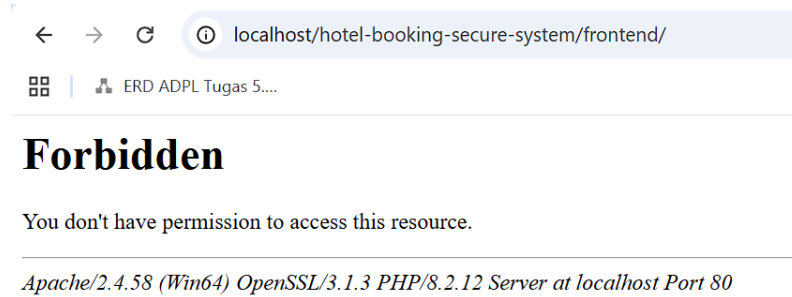
Berdasarkan hasil pengujian OWASP ZAP, terdapat tiga temuan prioritas yang perlu diperbaiki, yaitu .env Information Leak, Directory Browsing, dan Missing Security Headers. Perbaikan dilakukan dengan menambahkan file .htaccess pada root project.

File .htaccess digunakan untuk memblokir akses langsung ke file .env, menonaktifkan directory listing, dan menambahkan security headers pada file statis. Security headers yang ditambahkan meliputi Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, dan Permissions-Policy.



Gambar 2.13 Hasil Perbaikan .env Information Leak

Berdasarkan Gambar 2.13, file .env tidak lagi dapat diakses melalui browser. Dengan demikian, temuan .env Information Leak dinyatakan Fixed.



Gambar 2.14 Hasil Perbaikan Directory Browsing

Berdasarkan Gambar 2.14, folder /frontend/ tidak lagi menampilkan daftar file. Hal ini menunjukkan bahwa konfigurasi Options -Indexes pada .htaccess berhasil menonaktifkan directory browsing.

Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline' https://app.sandbox.midtrans.com https://cdnjs.cloudflare.com; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://cdnjs.cloudflare.com; font-src 'self' https://fonts.gstatic.com https://cdnjs.cloudflare.com; img-src 'self' data; connect-src 'self' https://app.sandbox.midtrans.com;
Date	Mon, 08 Jun 2026 08:12:10 GMT
Etag	"1ffb-653a51b170617"
Keep-Alive	timeout=5, max=100
Last-Modified	Sun, 07 Jun 2026 07:47:29 GMT
Permissions-Policy	camera=(), microphone=(), geolocation=()
Referrer-Policy	strict-origin-when-cross-origin
Server	Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN

Gambar 2.15 Hasil Perbaikan Security Headers pada File Statis

Berdasarkan Gambar 2.15, security headers telah muncul pada response file statis seperti login.html. Dengan demikian, temuan Missing Security Headers dinyatakan Fixed.

Tabel 2.5 Hasil Perbaikan Temuan

ID	Temuan	Sebelum Perbaikan	Setelah Perbaikan	Status
VULN-001	.env Information Leak	File .env dapat diakses	Akses ditolak/403 Forbidden	Fixed
VULN-002	Directory Browsing	Folder /frontend/ menampilkan daftar file	Directory listing tidak muncul	Fixed

VULN-003	Missing Security Headers	Header belum muncul pada file statis	Header keamanan muncul	Fixed
VULN-004	Server Version Disclosure	Versi server terlihat	Belum diperbaiki	Planned Fix
VULN-005	Sub Resource Integrity Missing	CDN belum memakai SRI	Belum diperbaiki	Accepted Risk

Berdasarkan Tabel 2.5, tiga temuan prioritas telah diperbaiki. Sementara itu, temuan Server Version Disclosure dan Sub Resource Integrity Missing dicatat sebagai rencana perbaikan lanjutan karena tidak termasuk temuan prioritas yang menghambat keamanan utama aplikasi pada minggu ketiga.

2.12 CVSS dan Risk Rating

Penilaian risiko dilakukan menggunakan CVSS untuk menentukan tingkat keparahan setiap temuan. Penilaian dilakukan berdasarkan kemudahan eksploitasi, kebutuhan autentikasi, interaksi pengguna, serta dampak terhadap kerahasiaan, integritas, dan ketersediaan sistem.

Tabel 2.6 CVSS Temuan Keamanan

ID	Temuan	CVSS	Severity	Status
VULN-001	.env Information Leak	6.5	Medium	Fixed
VULN-002	Directory Browsing	5.3	Medium	Fixed
VULN-003	Content Security Policy Header Not Set	5.3	Medium	Fixed
VULN-004	Missing Anti-clickjacking Header	4.3	Medium	Fixed
VULN-005	X-Content-Type-Options Header Missing	3.7	Low	Fixed
VULN-006	Server Version Disclosure	3.1	Low	Planned Fix

VULN-007	Hidden File Found /server-status	5.3	Medium	Planned Fix
VULN-008	Sub Resource Integrity Missing	2.7	Low	Accepted Risk
VULN-009	Semgrep Echoed JSON Response	2.7	Low	Accepted Risk
VULN-010	Semgrep Database \$host Detection	0.0	False Positive	False Positive

Berdasarkan Tabel 2.6, temuan dengan risiko paling tinggi adalah .env Information Leak dengan severity Medium. Temuan ini menjadi prioritas karena dapat menyebabkan kebocoran credential. Setelah dilakukan perbaikan, temuan prioritas telah berstatus Fixed.

2.13 Vulnerability Report

Vulnerability report disusun untuk mendokumentasikan temuan keamanan secara profesional. Setiap temuan dicatat dengan ID, nama kerentanan, CWE, lokasi, langkah reproduksi, dampak, rekomendasi, CVSS score, dan status perbaikan.

Tabel 2.7 Vulnerability Report

ID	Nama Kerentanan	CWE	Lokasi	Dampak	Rekomendasi	Status
VULN-001	.env Information Leak	CWE-215	/.env	Credential sensitif dapat terbaca	Blokir akses .env melalui .htaccess	Fixed
VULN-002	Directory Browsing	CWE-548	/frontend/	Struktur file terlihat	Tambahkan Options - Indexes	Fixed
VULN-003	Missing CSP Header	CWE-693	File statis frontend	Perlindungan browser kurang	Tambahkan CSP melalui .htaccess	Fixed
VULN-004	Missing Anti-clickjacking Header	CWE-1021	login.html	Risiko clickjacking	Tambahkan X-Frame-Options	Fixed

VULN-005	X-Content-Type-Options Missing	CWE-693	login.html	Risiko MIME sniffing	Tambahkan X-Content-Type-Options	Fixed
VULN-006	Server Version Disclosure	CWE-497	Header Server	Versi server terlihat	Atur konfigurasi Apache	Planned Fix

Berdasarkan Tabel 2.7, vulnerability report telah mencatat temuan utama yang diperoleh dari SAST, DAST, dan manual testing. Setiap temuan telah diberikan status untuk menunjukkan apakah telah diperbaiki, masih direncanakan, atau diterima sebagai risiko sementara.

2.14 Source Code Hasil Perbaikan

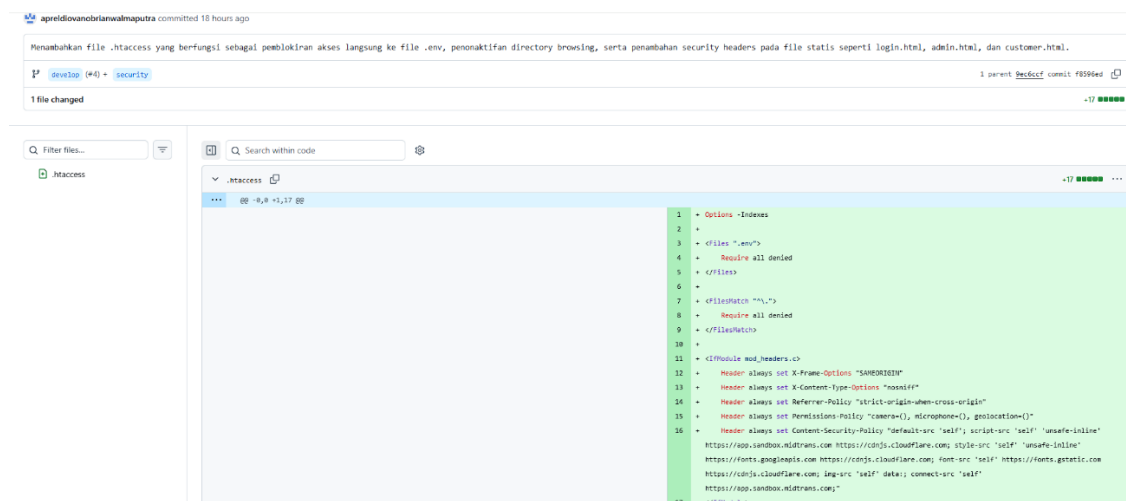
Perbaikan hasil security testing dilakukan dengan menambahkan file .htaccess pada root project. File ini digunakan untuk memperbaiki temuan .env Information Leak, Directory Browsing, dan Missing Security Headers.

Commit perbaikan dilakukan dengan perintah berikut:

`git add .htaccess`

`git commit -m "Menambahkan file .htaccess yang berfungsi sebagai pemblokiran akses langsung ke file .env, menonaktifkan directory browsing, serta penambahan security headers pada file statis seperti login.html, admin.html, dan customer.html."`

`git push origin payment`



Gambar 2.15 Commit Perbaikan .htaccess pada GitHub

Gambar 2.15 menunjukkan commit perbaikan yang dilakukan setelah proses security testing. Commit ini menjadi bukti bahwa hasil pengujian tidak hanya didokumentasikan, tetapi juga ditindaklanjuti dalam bentuk perubahan source code.

2.15 Executive Summary Postur Keamanan

Berdasarkan hasil pengujian minggu ketiga, postur keamanan sistem LuxeBooking mengalami peningkatan. Sistem telah diuji menggunakan SAST, DAST, dan manual security testing. Hasil pengujian menunjukkan bahwa beberapa kontrol keamanan telah berjalan dengan baik, seperti perlindungan terhadap SQL Injection, CSRF, XSS, IDOR, RBAC, session logout, dan cookie HttpOnly.

Temuan utama yang ditemukan adalah .env Information Leak, Directory Browsing, dan Missing Security Headers. Ketiga temuan tersebut telah diperbaiki melalui konfigurasi .htaccess. Setelah perbaikan, file .env tidak lagi dapat diakses, directory listing tidak lagi tampil, dan security headers telah muncul pada file statis.

Secara umum, sistem berada pada kondisi keamanan yang lebih baik dibandingkan sebelum pengujian. Namun, beberapa aspek masih perlu ditingkatkan pada minggu berikutnya, seperti mengurangi server version disclosure, membatasi akses /server-status, mengurangi penggunaan unsafe-inline pada CSP, dan mempertimbangkan Subresource Integrity pada resource CDN.

2.16 Hambatan dan Solusi

Selama proses pengujian minggu ketiga, terdapat beberapa hambatan yang ditemukan. Pertama, OWASP ZAP menemukan bahwa file .env dapat diakses langsung melalui browser. Hal ini terjadi karena file .env berada pada root project yang masih dapat dijangkau oleh Apache. Solusi yang dilakukan adalah menambahkan aturan pada .htaccess untuk memblokir akses terhadap file tersebut.

Kedua, folder /frontend/ masih menampilkan directory listing ketika diakses melalui browser. Kondisi ini dapat memperlihatkan struktur file aplikasi kepada pihak luar. Solusi yang dilakukan adalah menambahkan konfigurasi Options -Indexes pada .htaccess.

Ketiga, security headers belum muncul pada file HTML statis karena sebelumnya hanya diterapkan melalui config.php pada backend. Solusi yang dilakukan adalah menambahkan security headers melalui .htaccess, sehingga header juga berlaku pada file statis seperti login.html, admin.html, dan customer.html.

2.17 Rencana Minggu 4

Berdasarkan hasil pengujian dan perbaikan pada minggu ketiga, rencana kegiatan minggu keempat adalah melakukan hardening lanjutan terhadap aplikasi dan konfigurasi server. Beberapa rencana perbaikan yang akan dilakukan antara lain membatasi akses /server-status, mengurangi informasi versi server pada response header, mengurangi penggunaan unsafe-inline pada Content Security Policy, melakukan audit dependency, serta menyiapkan logging dan monitoring keamanan.

Rencana ini bertujuan untuk meningkatkan keamanan sistem secara bertahap dan menindaklanjuti temuan yang belum diperbaiki pada minggu ketiga. Dengan demikian, minggu keempat akan difokuskan pada penguatan konfigurasi, perbaikan lanjutan, dan peningkatan kesiapan sistem sebelum tahap finalisasi.

BAB III

KESIMPULAN

3.1 Kesimpulan

Pada minggu ketiga, sistem LuxeBooking telah melalui tahap pengujian keamanan yang mencakup SAST, DAST, manual security testing, CVSS scoring, dan vulnerability report. Pengujian SAST dilakukan menggunakan Semgrep untuk menganalisis source code, sedangkan pengujian DAST dilakukan menggunakan OWASP ZAP terhadap aplikasi yang berjalan pada localhost.

Hasil pengujian menunjukkan bahwa sistem telah memiliki beberapa kontrol keamanan yang berjalan dengan baik, seperti perlindungan terhadap SQL Injection, rate limiting login, CSRF protection, session logout, Role-Based Access Control, IDOR protection, cookie HttpOnly, dan XSS prevention. Pengujian manual menunjukkan bahwa sebagian besar fitur keamanan inti yang diimplementasikan pada minggu kedua berhasil berjalan sesuai tujuan.

Namun, OWASP ZAP menemukan beberapa temuan penting seperti .env Information Leak, Directory Browsing, dan Missing Security Headers. Temuan tersebut telah diperbaiki dengan menambahkan file .htaccess pada root project. Setelah perbaikan, file .env tidak lagi dapat diakses, directory browsing berhasil dinonaktifkan, dan security headers telah muncul pada file statis.

Dengan demikian, progress minggu ketiga telah berhasil memenuhi tujuan utama security testing, yaitu menemukan, menganalisis, memberi skor risiko, mendokumentasikan, dan memperbaiki temuan keamanan prioritas.

3.2 Saran

Berdasarkan hasil pengujian minggu ketiga, terdapat beberapa saran untuk pengembangan minggu berikutnya. Pertama, sistem perlu mengurangi informasi versi server yang masih muncul pada response header. Kedua, akses terhadap endpoint seperti /server-status perlu dibatasi atau dinonaktifkan. Ketiga, penggunaan unsafe-inline pada Content Security Policy dapat dikurangi pada tahap hardening berikutnya.

Selain itu, sistem juga dapat ditingkatkan dengan melakukan audit dependency, menambahkan logging keamanan, melakukan scan ulang OWASP ZAP setelah seluruh perbaikan diterapkan, dan mendokumentasikan hasil hardening pada laporan minggu

keempat. Dengan langkah tersebut, keamanan sistem LuxeBooking dapat terus ditingkatkan secara bertahap.